

Proyecto Integrador

**Procesamiento de Lenguaje Natural (PLN)
Detección de Tema en Chats De Atención
Ciudadana del RUTS**

Primer Corte

Mtro. Pablo Ricardo Sanchez Gomez

Jessica Melani Romero Lora (253220116)

Víctor Manuel Santos Martínez (253220020)

28 de mayo del 2026



Tabla de contenido

1. Fundamento técnico.....	3
2. Resultados Centrales.....	7
3. Conclusiones.....	8
4. Reflexiones.....	9
5. Aplicaciones futuras y potencial metodológico.....	10

1. Introducción

En esta tarea se desarrolló un pipeline de Procesamiento de Lenguaje Natural (PLN) aplicado a una base de datos proveniente de la Subdirección de Atención Ciudadana del RUTS (Registro Único de Trámites y Servicios del Estado de Hidalgo), la cual recibe a diario consultas por chat sobre trámites de gobierno diversos, tales como: actas del registro civil, placas vehiculares, predial e impuestos, becas, avisos sanitarios, etc. Hoy esas conversaciones se atienden y archivan sin una clasificación temática sistemática.

Durante el desarrollo del proyecto, se eligió optar por detección del tema a partir de una conversación ciudadana, con el objetivo de asignarle la categoría del trámite. En este sentido, el presente documento presenta un problema de clasificación de texto en español, sobre lenguaje real, informal y abreviado.

Esta base de datos y enfoque son adecuados para el desarrollo de la práctica de Procesamiento de Lenguaje Natural (PLN), porque los chats corresponden a texto libre en español, con ruido (faltas de ortografía, mayúsculas, datos personales) y una intencionalidad comunicativa implícita que debe inferirse del contenido, exactamente el tipo de problema que la representación y clasificación de texto resuelve.

Durante este proyecto se pretende alcanzar los objetivos propuestos, tales como definición del problema, construcción del corpus inicial, limpieza/normalización, tokenización y exploración. La vectorización y el modelado corresponden al segundo corte de evaluación. La fuente de datos seleccionada es un export de 2,014 conversaciones equivalentes a 31, 600 mensajes del chat del RUTS, obtenidos vía COEMERE. Los datos crudos contienen información personal y se procesan fuera del repositorio, al corpus solo se integra texto anonimizado (cumplimiento LGP DPSO).

2. Fundamento técnico

En la primera etapa de diseño del pipeline se cargaron las librerías base y la lógica reproducible en [src/pipeline.py](#)

```
import sys
from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Importamos el pipeline del proyecto (lógica reproducible en src/pipeline.py)
PROJ = Path.cwd().parent
sys.path.append(str(PROJ / "src"))
import pipeline as pl

sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 110
FIG = PROJ / "reports" / "figures"
FIG.mkdir(parents=True, exist_ok=True)
print("Pipeline cargado. Fuente cruda:", pl.RAW_MENSAJES.name)
```

Python

Posteriormente, en la etapa de construcción del corpus y anonimización de datos personales, se procesó cada conversación en un documento concatenando únicamente los mensajes del visitante (input), es decir, las palabras del ciudadano. Previa a la integración al corpus se aplicó el proceso de anonimización conforme al reglamento LGP DPSO. En este sentido se identificaron los marcadores principales como nombre, e-mail, CURP, RFC, teléfono y número celular.

Marcador	Elemento reemplazado
Nombre	Nombre de usuario
E-mail	Correos electrónicos
CURP/ RFC	Claves CURP y RFC
Teléfono y Número	Teléfonos y folios / Números largos

Durante esta clasificación se descartaron los chats sin preguntas reales (saludos/ agradecimientos sueltos): se exige un mínimo de 3 tokens con contenido.

Posteriormente, se construye el corpus desde los mensajes crudos (un documento por chat).

```
corpus = pl.construir_corpus()
print(f"Documentos en el corpus: {len(corpus)}")
corpus.head(5)[["id", "texto", "etiqueta", "n_tokens"]]
```

Python

Durante la fase de anonimización se propone la siguiente estrategia con algunos ejemplos de datos personales redactados, también se realizó una verificación donde se evitó mantener PII estructurada (correos, URLs, números largos).

```
mask = corpus["texto"].str.contains(r"\[NOMBRE\]|\[EMAIL\]|\[RFC\]|\[TEL\]", regex=True)
for t in corpus.loc[mask, "texto"].head(4):
    print("•", t[:160])

fuga = corpus["texto"].str.contains(r"@w|https?:\/\/|b\d{7,}\b", regex=True).sum()
print(f"\nFilas con PII estructurada residual: {fuga}")
```

Python

3. Resultados Centrales

4. Conclusiones

5. Reflexiones.Aplicaciones

futuras y potencial
metodológico